CLOUD COMPUTING PROJECT · PROJECT GUIDE

Behavioural Fraud Detection for Digital Banking

The whole project in simple language: what it does, how it works, why it uses simple models instead of heavy machine learning, what it achieves, and where it is weak.

Written in the style of ASD-STE100 Simplified Technical English
October 2026

Contents

This guide explains the full project to a person who knows nothing about it. It uses the rules of ASD-STE100 (Simplified Technical English) as far as possible: short sentences, active voice, one topic in each paragraph, and one meaning for each word. Technical names (for example "DynamoDB") stay as they are, because you need them to find the parts in the code.

For the full design, read docs/architecture.md in the project repository. For all the measured numbers, read docs/results.md. This guide is the short and simple version of both.

1. Words you must know
2. The problem
3. What this system does, in one page
4. The attacks that the system stops, and the attacks it does not
5. What happens when a customer sends money
6. The five signals
7. How the system makes one decision
8. What happens after the decision
9. The slow work that the system does every night
10. How a profile learns, and the poisoning attack
11. Privacy and safety rules
12. The cloud parts and why each part is necessary
13. Why we do not use a heavy machine learning model
14. Tradeoffs
15. Results
16. Why this project is good
17. Where this project is weak
18. Work for the future
19. How the code is arranged
20. Summary

1. Words you must know

Word	Meaning in this project
Behavioural biometrics	Identification of a person from how they do a thing, not from what they know. Here, the thing is typing.
Keystroke dynamics	The timing of the keys when a person types. For example, how long a key stays down, and the time between two keys.
Hold time	The time from "key goes down" to "key comes up".
Flight time	The time from one key to the next key.
Profile	A record of how one person types. It holds a "centre" (the usual value of each timing feature) and a "scale" (how much that value usually changes).
Session	One visit to the bank website, from sign-in to the end of a transfer.
Checkpoint	A point in a session where the system calculates risk. There are four: after login, after the payee, after the amount, and at confirmation.
Payee	The person or account that receives the money.
Channel	One type of evidence. The system has five channels: behaviour, automation, transaction, context and payee.
Risk score	A number from 0 to 100. A high number means a high chance of fraud.
Step-up	The system asks the customer to prove their identity again with a passkey before the money moves.
Passkey	A sign-in method that uses a key stored in the customer's own device (phone, laptop, security key). It replaces a password. A thief cannot easily copy it.
Friction	Extra work that the system gives to a genuine customer, for example a step-up. Less friction is better.
Adaptation	The profile changes slowly over time, because the way a person types changes over time.
Drift	A slow, genuine change in how a person types.
Poisoning	An attack on adaptation. The attacker teaches the profile to look like the attacker.
Serverless	A cloud design with no servers that you start, stop or patch. The cloud runs your code only when a request arrives, and you pay only for that work.
Lambda	The AWS service that runs our code in a serverless way. One "Lambda" is one function.
Fail open	When a part breaks, the system lets the customer continue (with monitoring). It does not block the customer.
EER	Equal Error Rate. The error rate at the point where "wrong person accepted" and "right person rejected" are equal. Lower is better.
p50, p95	Latency measures. p50 is the typical time. p95 is the time that 95 out of 100 requests are faster than.

2. The problem

Banks check a customer's identity one time, at login. They use a password, an OTP, or a device check. After this check, the bank trusts the session until it ends.

This is not sufficient. An attacker can steal a password with phishing or malware. An attacker can take an OTP with a SIM swap. When the attacker passes the login check, the bank sees a "genuine" customer. The bank cannot see that a different person now types on the account.

There are four specific gaps:

1. **Identity is checked one time, not all the time.** A takeover after login is not visible.
2. **Detection is often late.** Many systems find the fraud after the money has gone.
3. **Decisions are "yes" or "no", with no reason.** A genuine customer gets blocked and nobody can say why. An analyst has no information to act on.
4. **Profiles that learn can be attacked.** A typing profile must change over time, or the system rejects genuine customers whose typing has changed. But if the profile learns from sessions that "look genuine", an attacker can send sessions that look just genuine enough. Slowly, the profile moves toward the attacker. Research papers recommend adaptive profiles, but they do not measure this risk.

Gap 4 is the research question of this project.

3. What this system does, in one page

The system is a fraud check for a simulated net banking website. It runs on Amazon Web Services (AWS).

When a customer makes a transfer, the website records **only the timing of the keys**. It does not record which keys. It does not record what the customer types. It sends these timings to the cloud at four checkpoints.

In the cloud, one function collects five types of evidence:

1. **Behaviour.** Does this typing match this customer's typing profile?
2. **Automation.** Does this typing look like a robot, or like a copy of an old session?
3. **Transaction.** Is this amount, time or speed of transfers unusual for this customer?
4. **Context.** Is this a new device? Did the password or phone number change recently?
5. **Payee.** Is this payee new? Do many unrelated people send money to this payee?

The function combines the five into one risk score from 0 to 100. It also gives the three largest reasons for the score, and a confidence value. Then it selects an action:

Action	What happens
allow	The money moves.
monitor	The money moves. The system watches more carefully.
step-up	The money waits. The customer must confirm with their passkey.
restrict	The money waits. A bank analyst must look at it.
block	The transfer is cancelled. An alert goes to the bank.

The money moves only when the transfer is "released". A separate workflow controls this. A mock ledger (a simple bank balance) changes only on release.

The project also asks a research question: **how can a typing profile learn safely, so that an attacker cannot poison it?** The project tests four update policies on real typing data and reports what works and what does not.

4. The attacks that the system stops, and the attacks it does not

4.1 In scope

Attack	What the attacker does	What the system sees
A. Account takeover	The attacker has the password and uses the account.	Different typing, a new device, a new payee, an unusual amount.
B. Bot and replay	A script types for the attacker, or the attacker plays back a recording of the real customer's typing.	Timing that is too regular, or timing that matches an old session exactly.
C. Low-and-slow siphoning	The attacker sends many small transfers over days or weeks. No single transfer looks bad.	The total over time looks bad: a new payee, regular gaps, amounts just below limits, and typing that is "a bit wrong" only on transfers to that payee.
D. Profile poisoning	The attacker sends many sessions to make the profile learn the attacker's typing.	This is an attack on the system itself. Section 10 explains it.

4.2 Out of scope

The system does **not** try to detect:

- Scams and social engineering, where a criminal tricks the real customer into sending money.
- Coercion, where somebody forces the real customer to send money.
- The intention of the customer.

Why: in these cases, the genuine customer types. Typing evidence cannot help. These problems need different tools (for example, warnings to the customer), and they are a different project. We keep the scope small so that we can measure it correctly.

5. What happens when a customer sends money

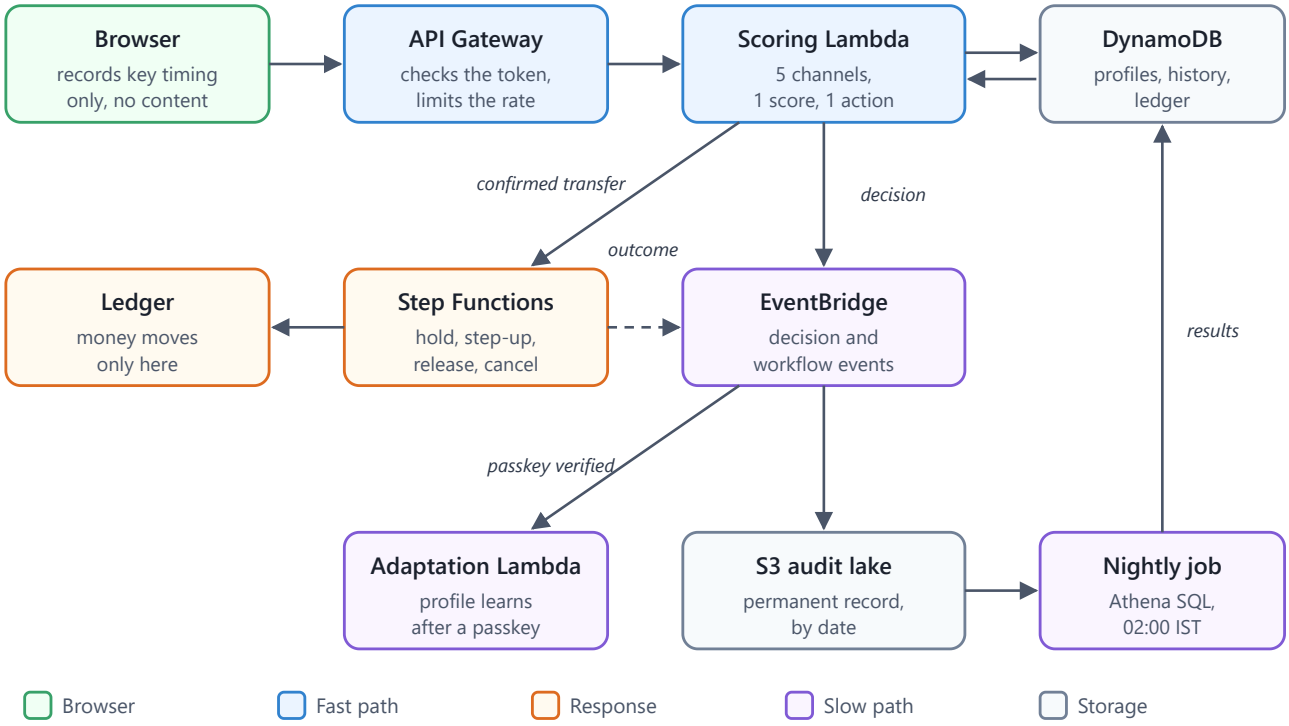


Figure 1. The path of one transfer through the system.

Step by step:

1. The customer signs in with a passkey. Amazon Cognito checks it and gives the browser a token.
2. The customer opens the transfer form. The browser records the timing of each key press.
3. At each checkpoint, the browser sends the timings and the token to API Gateway.
4. API Gateway checks the token. It also limits how many requests one user can send.
5. The scoring Lambda reads all the data it needs from DynamoDB in **one** read. This data is the profile, the known devices, the payee history, the payee risk and the customer's normal amounts.
6. The scoring Lambda calculates the five channels, the risk score, the reasons and the action.
7. It saves the decision and sends a "decision event" to EventBridge.
8. At the confirmation checkpoint, it starts a Step Functions workflow for the transfer. It does not wait for the workflow. It replies to the browser immediately.
9. The workflow releases the money, or holds it for a step-up or an analyst, or cancels it.
10. Every event goes to an S3 "audit lake" (a permanent record).
11. Each night, a batch job reads the lake and calculates long-term patterns. It writes the results back to DynamoDB, so that the next transfer can read them immediately.
12. When a customer passes a passkey step-up, the adaptation Lambda can update their typing profile.

Only the confirmation checkpoint can restrict or block. The earlier checkpoints can only raise a step-up.

6. The five signals

6.1 Behaviour (who types)

The browser records three lists of times for each field: hold times, down-to-down times and up-to-down times. It also counts backspaces, corrections and pastes.

From these, the system calculates 12 features:

Feature	Meaning
mean hold, std hold	Average hold time, and how much it changes
mean flight, std flight	Average time between keys, and how much it changes
burst count	Number of very fast key pairs (under 50 ms)
pause count, longest pause	Number of pauses over 0.5 s, and the longest pause
entry duration	Total time for the field
typing speed	Keys per second
backspace rate, correction rate, paste count	Edits. These three are only for live use, because the research dataset has no edits.

The profile stores a centre and a scale for each feature. The system compares the session to the profile with the **scaled Manhattan distance**:

```
score = sum over all features of  |session value - profile centre| / profile scale
```

In simple words: for each feature, find the difference, divide it by the normal variation, and add all the results. A big total means "this typing is not like the usual typing".

Each customer has a different profile for each **device class** (desktop keyboard, phone, tablet). People type differently on a phone and on a laptop. If we used one profile, a change of device would look like an attack.

6.2 Automation (robot or copy)

- **Regularity.** Humans do not type with perfectly equal gaps. A script often does. The system measures the coefficient of variation (CV) of the gaps. A human CV is well above 0.2. A simple bot is near 0.
- **Replay.** The system keeps short "fingerprints" (hashes) of the timing of the last 50 confirmed sessions. If a new session has the same timing pattern, it is a copy. The hashes contain only rhythm, never a key.

6.3 Transaction (what is sent)

The system compares the amount, the hour of day and the number of transfers today with this customer's own history (for example, their 95th percentile amount). Only "more than usual" counts. A small transfer does not lower the risk.

6.4 Context (from where)

Is the device new? Did the password, phone number or email change in the last days? A new device plus a new password is the typical pattern of a takeover.

6.5 Payee (to whom)

- Is this payee new for this customer?
- Did this customer ever verify this payee with a passkey?
- From the nightly job: do many unrelated people send money to this payee? (This is the pattern of a "mule" account that collects stolen money.)
- From the nightly job: does this customer show a siphoning pattern to this payee?

6.6 Confidence

Each channel also gives a **confidence** from 0 to 1. Confidence means "how much evidence do we have", not "how sure are we of guilt".

Examples:

- A profile with only 2 sessions gives low confidence.
- A short field with 5 keys gives low confidence.
- A new customer with no transfer history gives zero confidence on the transaction channel.

Low confidence makes the channel count less. This stops a weak signal from blocking a genuine customer.

7. How the system makes one decision

7.1 The combination (fusion)

The system uses **logistic regression**. This is a simple, well-known formula:

```
for each channel:   z = confidence x (score - normal mean) / normal scale
L    = intercept + sum of (weight x z)
risk = 100 x 1 / (1 + e^(-L))
```

There are only 5 weights and 1 intercept. A research script fits them from data, and the result is in `src/fraudcore/models.json`. The script also forces each weight to be zero or more. A channel must never make the risk lower when it sees something strange.

Because the formula is a simple sum, the reason for a score is exact: each channel's part is `weight x z`. The system reports the three largest parts. Nobody has to guess why a score is high.

7.2 From score to action

The thresholds come from the genuine sessions in the training data:

Risk score	Action
below 23.2	allow
23.2 and more	monitor
36.5 and more	step-up
63.0 and more	restrict
79.5 and more	block

7.3 Three rules that the model cannot change

The model learns from partly synthetic data, so we do not give it full power. Three fixed rules can only make the action **softer**, never stronger:

1. **Friction ceiling.** Typing evidence alone (behaviour and automation) can never go above step-up. To restrict or block, at least one other channel (transaction, context or payee) must also alert. Result: if a genuine customer's typing changes, the worst thing that happens is one passkey check.
2. **Corroboration for block.** A block needs two channels that alert, or one channel plus a flag from the nightly job. Otherwise the action stops at restrict.
3. **Checkpoint rule.** Only the confirmation checkpoint can restrict or block.

There is also a **fail-open rule**. If DynamoDB or another part fails, the system returns "monitor" with zero confidence. A database problem must never lock out a customer.

Each of these rules has a unit test that fails if somebody removes the rule.

8. What happens after the decision

A Step Functions workflow controls every confirmed transfer. There is only **one** path to the ledger, so the rule "money moves only on release" is in one place.

Action	Workflow path
allow, monitor	Release immediately.
step-up	Hold. The customer sees a passkey dialog. If Cognito accepts the passkey, release. If time runs out, cancel.
restrict	Hold. An analyst sees the transfer in the operations console and selects "release" or "deny".
block	Cancel. SNS sends an alert.

The step-up check happens on the server. The browser cannot just say "the customer passed". The server asks Cognito for a challenge, and only a correct passkey answer releases the transfer.

When a step-up passes, the workflow sends a `stepup.verified` event. This event is the **only** thing that can start a profile update (section 10).

An analyst's release is different. It releases one transfer, but it does not verify the payee and it never updates the profile.

9. The slow work that the system does every night

Some fraud is not visible in one session. Low-and-slow siphoning is an example. To see it, the system must look at weeks of data.

1. Every decision and every workflow result goes to EventBridge, then to an S3 bucket. The files are in folders by date. This is the **audit lake**.
2. At 02:00 India time, a scheduler starts the **aggregator** Lambda.
3. The aggregator runs one Athena SQL query over the last 90 days of released transfers.

4. Pure Python code in `fraudcore/batch.py` calculates: - the customer's normal amounts, counts and hours; - for each new payee: total money sent, how regular the gaps are, how many amounts are just below the customer's high-amount limit, and whether the typing on those transfers is worse than the typing on other transfers; - for each payee across all customers: how many different senders, and how many are new.
5. The results go back to DynamoDB. The next live transfer reads them by key, with no extra calculation.

Important rule for siphoning: a regular payment to a new payee is also what a genuine customer does when they pay a new tutor or a new landlord. So the system flags a payee only if **identity** also agrees: the typing on transfers to that payee must be at least 30% further from the profile than the typing on transfers to other payees. This is the sign of a second person on the account.

Batch results become old. If the nightly job fails, its evidence fades to zero over one week. The system never keeps an old verdict forever.

This detection is **after the fact** by design. Some money always moves before the flag.

10. How a profile learns, and the poisoning attack

This is the research part of the project.

10.1 Why a profile must learn

People change how they type. They get faster on a password they use every day. They buy a new keyboard. If the profile never changes, the system rejects more and more genuine sessions. On real data, a profile that never changes rejects **29%** of the person's later typing.

10.2 Why learning is dangerous

The simple method is: "if a session looks genuine, move the profile a little toward it." This is an exponentially weighted mean (EWMA).

This method has two faults:

1. The system uses its own judgement to decide what to learn. The attacker can make sessions that pass that judgement on purpose.
2. A mean has no protection against bad points. One far point moves it. Many far points move it without limit.

So the attacker sends sessions that are "just genuine enough", each a little closer to their own typing. Over time, the profile becomes the attacker.

10.3 The four policies we compare

Policy	What it learns from	How it calculates the profile	Limit on movement
P0 static	nothing	fixed at enrolment	none
P1 naive	sessions that look genuine	EWMA	none
P2 trust-gated	sessions with high trust	EWMA	none
P3 proposed	sessions with high trust	weighted geometric median	yes, a displacement budget

P2 lets us measure the trust gate alone. P3 minus P2 lets us measure the median and the budget.

10.4 The parts of P3

Trust gate. Trust comes from evidence **outside** the typing channel. If typing decided what typing learns, the attacker could control it. Trust is a product of four gates:

```
trust = verify x device x stability x transaction
```

- verify: 1.0 for a passkey step-up, 0.6 for a passkey sign-in, 0.3 for password only;
- device: 1.0 for a device with 10 or more clean sessions, 0.5 for a younger known device, 0 for a new device;
- stability: 0 if the password, phone, email or passkey changed in the last 7 days, otherwise 1;
- transaction: lower if the non-typing channels alerted.

We multiply, and do not add, because each gate must be able to say "no" alone. A new device plus a new password is a takeover, and nothing else can outvote that. A session with trust below 0.5 is not used at all. (A small weight times many sessions is still a large effect.)

Robust centre. The profile centre is the **weighted geometric median** of the last 200 trusted sessions, not the mean. The median has a "breakdown point" of 0.5. This means: if an attacker controls less than half of the weight, they cannot move the centre an unlimited distance, however extreme their sessions are. The mean has a breakdown point of 0.

Displacement budget. The profile can move only a limited distance from an "anchor" (the last verified good profile). The limit comes from real data: how much genuine people drift between two recording days. If the new centre is too far, the system moves it only to the edge of the limit and records a `budget_saturated` metric. A genuine person rarely hits the limit. An attacker who moves as fast as possible hits it all the time.

Re-anchor. The anchor moves forward only after 3 passkey step-ups on 3 different days, or after 30 quiet days.

Frozen scale. The profile's scale (the normal variation) does not adapt at all. Section 10.6 explains why.

10.5 The data and the method

We use the CMU keystroke dataset (Killourhy and Maxion, 2009). It has 51 people. Each person typed the same 10-character password 400 times, over 8 sessions on different days. It is public, the people gave consent, and it has a published error rate that we can compare with.

For each victim, we use their early days to build the profile, the middle days as genuine drift, and the last days as a test. Other people in the dataset act as attackers. We run 51 victims, two attackers each, five random seeds per pair, and we report 95% confidence intervals.

"Impersonation" is the share of the attacker's normal typing that the final profile accepts. "False rejection" is the share of the victim's later typing that the final profile rejects.

10.6 Results (on the setup that is deployed)

Policy	Impersonation	False rejection
P0 static	46%	29%
P1 naive EWMA	86%	19%
P2 trust-gated EWMA	82%	12%
P3 with adaptive scale	100%	0%
P3 with frozen scale (adopted)	74.5%	13%

What this shows, in simple words:

1. **Naive learning is easy to poison.** Impersonation goes from 46% to 86%.
2. **A profile that never learns rejects many genuine sessions** (29%). So the tradeoff is real.
3. **If the scale can learn, the attacker wins every time.** The attacker makes the "normal variation" very large, and then everything looks normal. So the scale must be frozen.
4. **With a frozen scale, P3 is about 8 points better than P2.** The confidence intervals overlap, so this gain is small and not certain.
5. **The budget does not limit the attacker in the deployed setup.** In the deployed system, only passkey step-ups can update a profile. An attacker who has the passkey also re-anchors the profile often, so the budget never fills.
6. **Learning costs some security even with no attacker.** P3 with zero attacker step-ups still accepts 60% of the attacker, compared to 46% for a static profile.

The honest conclusion: what stops poisoning in the deployed system is the **passkey step-up gate**. An attacker without the customer's passkey cannot change any profile. (In the live attack test, 0 profiles changed.) An attacker who has the passkey can still poison the profile, and the median and budget give only a small extra defence against that attacker.

11. Privacy and safety rules

These rules are part of the design. Each rule has a test.

- **No typed content leaves the browser.** The browser uses the key name only in memory, to match "key down" with "key up". The payload contains only time values and three counts. No characters, no account numbers, no amounts. The payee account goes only as a SHA-256 hash.
- **Typing profiles are personal data.** They are encrypted with a customer-managed KMS key.
- **Least privilege.** Each Lambda has its own IAM role with only the access it needs. Only the adaptation Lambda can write a profile. The scoring Lambda cannot delete a profile.
- **Data is deleted on time.** Sessions after 1 day, decisions after 30 days, trusted sessions after 180 days. Old lake files move to cheaper storage after 90 days.
- **Fail open.** A broken part gives "monitor", never "block".
- **Friction ceiling.** Typing alone can never block a customer.
- **No VPC, no servers that run all the time.** Section 12 explains why.

12. The cloud parts and why each part is necessary

All infrastructure is written in Terraform, in the `ap-south-1` (Mumbai) region. The full stack can be created from empty and destroyed cleanly.

AWS service	What it does here	Why we need it
S3 + CloudFront	Hosts the banking website. Stores the audit lake.	Cheap, fast static hosting. A permanent, append-only record for batch analysis.
Cognito	Passkey sign-in and passkey step-up.	The whole trust model depends on a device-bound key. SMS OTP fails against SIM swap.
API Gateway (HTTP API)	The one public entry point. Checks the token. Limits request rate.	Unauthenticated or flooding traffic never reaches the scoring code.
Lambda	Runs all our code (scoring, adaptation, aggregator, ledger, archive, transfers, console).	Runs only when needed, costs nothing when idle.
DynamoDB	One table for profiles, devices, payees, decisions, ledger.	Reads in a few milliseconds. On-demand price, no idle cost, no VPC.
Step Functions	The transfer workflow: hold, step-up, timeout, release, cancel.	A transfer can wait minutes for a passkey. A Lambda cannot wait that long cheaply.
EventBridge	Sends decision events to the lake, to adaptation and to alerts. Runs the nightly schedule.	New consumers do not need any change to the scoring code.
Athena	SQL over the audit lake for the nightly job.	Batch questions are SQL questions. Scans of DynamoDB are slow and expensive.
SNS	Alert on block and restrict.	Simple alerting.
KMS	Encrypts profiles and the lake.	A key policy that we control.
CloudWatch + X-Ray	Logs, metrics, latency breakdown for each part.	Latency is a result of the project, so we must measure each part.
Budgets	A 5 USD cost alarm.	The attack simulator can create real cost.

Services we do not use, and why:

Not used	Reason
EC2, ECS, RDS, OpenSearch	They cost money when idle.
VPC, NAT gateway	No part needs a private network. A VPC adds cold-start time, and a NAT gateway costs money every hour.
SageMaker endpoint	The model is 6 numbers. A hosted endpoint costs money every hour and adds network time.
Kinesis, Firehose	Built for very high data rates. We have a few requests per second.
Graph database (Neptune)	The payee questions are counts and sums, not graph searches.
Amazon Fraud Detector	Closed to new customers, and it would replace the part that is our research.

Not used	Reason
Bedrock or any LLM	Out of scope. Our explanations are exact without an LLM.

13. Why we do not use a heavy machine learning model

"Heavy model" here means a deep neural network, a large gradient-boosted model, or an LLM. We use simple, well-understood methods instead. There are seven reasons.

13.1 We do not have the data

Only the typing channel has real data (51 people from CMU). No public dataset joins real typing with real bank transactions for the same people. So the data for the transaction, context and payee channels comes from a simulator that we wrote.

A model with a lot of capacity learns the patterns in its training data. If the training data comes from our simulator, a deep model learns **our simulator**, not fraud. Its results would look very good and mean nothing. A simple model with 6 numbers cannot learn the small accidents of the simulator.

13.2 The simple method is the best published method on this dataset

In the published CMU benchmark, the **scaled Manhattan distance** is the best of the detectors that the authors compared. We reproduced their result exactly (9.62% EER). With a different method, we could not compare our numbers to published work.

13.3 Security needs guarantees that a neural network does not give

The geometric median has a known breakdown point of 0.5. We can say exactly how much of the data an attacker must control to move it. For a learned neural encoder, nobody can make that statement. For a research question about poisoning, this guarantee is the main point.

13.4 Exact explanations

With logistic regression, the reason for a score is exact: `weight x z` for each channel. This is the same answer that SHAP (a popular explanation tool) gives for a linear model, but with no extra library and no estimate. An analyst sees exactly why. A deep model needs approximate explanations.

13.5 Speed and cold start

The Lambda uses only the Python standard library and boto3. There is no numpy, pandas or scikit-learn in the cloud code. The package is small, so a cold start is faster, and the warm handler takes about 24 ms. A deep learning library adds hundreds of megabytes and makes cold starts much slower.

13.6 Cost

The model is 6 numbers in a JSON file. It costs nothing to "host". A model server (for example a SageMaker endpoint) costs money every hour, even with no traffic. Our total measured cost is 0.23 USD per thousand sessions.

13.7 Audit and control

A bank or an examiner can read 5 weights and 4 thresholds. They can check that no channel lowers the risk. They can check the fixed rules. With a black-box model, this is not possible.

13.8 What we lose

A heavy model is not always wrong. With a lot of real data, it can do better:

- Deep models on large free-text typing datasets (thousands of people) can identify people better than our 9 summary features.
- A model can find combinations of signals that a linear sum cannot express.

We do not have that data, so we cannot get that benefit honestly. The design keeps this option open: the research folder can test a heavier model as a comparison, and if it wins on synthetic data, that is a statement about the simulator, not about fraud.

13.9 Where we use each method

Stage	Method	Why not something heavier
Typing features	12 fixed formulas	Learned features need much more data.
Profile	Weighted geometric median	Must resist poisoning, with a known guarantee.
Typing score	Scaled Manhattan distance	Best published detector on this dataset.
Bot and replay	CV of gaps, timing hashes	Bots fail on regularity. A classifier needs bot labels.
Transaction	Per-customer percentiles	A model for each customer is not possible with little data.
Context and payee	Counts and ratios	These are facts, not patterns.
Combination	Logistic regression, 5 weights	Exact reasons, small, auditable.
Two typists on one account	k-medians clustering, offline	Clustering is the correct tool for "one group or two".

14. Tradeoffs

Each design choice gives something and costs something. This table lists the important ones.

We chose	Instead of	What we get	What we pay
9 summary typing features	31 per-key timings	Works on any field, not only one fixed password.	Error rate doubles: EER 20.4% instead of 9.62%.
Logistic regression	A deep or boosted model	Exact reasons, tiny model, no idle cost, no overfit to the simulator.	Cannot learn complex combinations.
Typing alone stops at step-up	Let typing block	A genuine customer whose typing changes gets at most one passkey check.	A takeover with perfect context (known device, known payee) gets only a step-up.
Fail open	Fail closed	No customer is locked out by a database problem.	During a failure, fraud can pass with only monitoring.

We chose	Instead of	What we get	What we pay
Learn only from passkey step-ups	Learn from every "genuine-looking" session	An attacker without the passkey cannot touch the profile.	The profile learns slowly. If the attacker has the passkey, they can still poison it.
Frozen profile scale	Adaptive scale	Stops the "make everything look normal" attack.	The profile cannot adjust if a person's natural variation changes.
Serverless, no VPC	Servers or containers in a VPC	Zero idle cost, no patching, faster cold starts.	Cold starts of about 1.2 s. Less control of the network.
Nightly batch for siphoning	Real-time stream processing	Cheap and simple. SQL that a reader can check.	Detection is after the fact. Some money moves before the flag.
Siphoning flag needs identity evidence	Flag on amount and timing patterns only	Genuine customers who pay a new tutor are not restricted.	A siphoner whose typing is similar to the victim is missed.
Profile per device class	One profile per customer	A change from laptop to phone is not an "attack".	Each new device class needs 5 sessions before it has full confidence.
Every transfer goes through Step Functions	Release "allow" transfers directly	One path to the ledger; one place to enforce "money moves only on release".	Step Functions is 46% of the cost per session, and adds time at confirmation.
Pure Python in the cloud	numpy in the cloud	Small packages, one implementation shared with research, zero runtime dependencies.	We write simple maths by hand and test it.

15. Results

There are two types of evidence. We never mix them.

- **Tier 1** comes from real data or real measurement: CMU typing, and the latency and cost of the deployed system. This is the real result.
- **Tier 2** comes from our own simulator. It shows that the logic does what we designed. It does not prove real-world detection rates.

15.1 Typing identity (Tier 1)

Method	Average EER over 51 people
Published baseline (Killourhy and Maxion, 2009)	9.62%
Our reproduction, same 31 per-key timings	9.62%
Our deployed features, 9 summary values	20.4%

15.2 Poisoning (Tier 1)

See section 10.6.

15.3 Speed (Tier 1, measured on the deployed system)

Measure	p50	p95
Warm round trip (browser to cloud and back)	145 ms	229 ms
Warm scoring code only	24 ms	29 ms
Warm DynamoDB read	3.8 ms	4.8 ms
Warm confirmation (starts the workflow)	172 ms	392 ms
Cold round trip	1,162 ms	1,500 ms

The target was p95 under 200 ms. The code meets it easily. The full round trip misses it (229 ms), because most of the time is API Gateway and the network.

15.4 Cost (Tier 1)

0.23 USD per thousand sessions, from real usage counts over 250 sessions and 1,000 API calls. Step Functions is 46% of this. Nothing costs money when the system is idle.

15.5 Attack detection on the live system (Tier 2)

10 victims, 50 sessions for each attack type.

Attack	Detected	Action
Account takeover	100%	block
Bot (equal gaps)	100%	step-up
Replay (with the protocol nonce turned off)	100%	step-up
Poisoning session	36%	step-up
Genuine customer	16% get friction	step-up

Bot and replay stop at step-up because of the friction ceiling. The 100% values come from simulated attacks that are easy to separate. Do not read them as real-world rates.

15.6 Siphoning (Tier 2)

20 simulated days. 5 victims send ₹900 every two days to one shared mule. 3 genuine customers pay a new payee on the same schedule.

- The nightly job flagged 4 of 5 victims, on day 5. ₹2,700 moved before the flag.
- 0 genuine customers were restricted or blocked.
- The missed victim is the one whose attacker typed similarly to them.

15.7 Two typists on one account (Tier 1)

Share of sessions from a second person	0%	10%	20%	30%	50%
Accounts flagged	0%	0%	24%	28%	32%

The detector never flagged a one-person account in the test. It finds about one quarter to one third of shared accounts. It is not part of the live system yet.

16. Why this project is good

1. **It is a real, deployed system, not only a notebook.** Most research papers in this area train a model on a dataset and stop. This project serves every decision live on AWS, and measures latency and cost. No paper in our 15-paper survey reports cost per thousand sessions.
2. **It asks a question that the literature does not ask well.** Papers recommend adaptive profiles but do not measure how easy they are to poison. This project builds the attack, builds a defence, and measures both.
3. **It reports honest results, including negative ones.** The budget does not bind in the deployed setup, and the document says so. The results changed when we found two problems in our first test method, and both versions are kept and explained.
4. **One copy of the decision logic.** All logic is in `src/fraudcore/`. The research scripts and the cloud functions import the same code. So the research numbers describe the deployed system.
5. **Safe by design.** Typing alone can never block a customer. A failure never blocks a customer. Typed content never leaves the browser. Only a passkey can update a profile. Each rule has a test that fails if somebody removes the rule.
6. **Every decision has a reason.** The top three reasons come with each score, and they are exact.
7. **Very cheap.** 0.23 USD per thousand sessions, and zero cost when idle.
8. **Reproducible.** Every number comes from a script with a fixed seed. Every figure and table is in the repository. The stack is created and destroyed by Terraform.
9. **Well tested.** A fast unit test suite runs with no AWS access. Each phase has an integration test that runs against the real deployed stack, and the earlier tests keep running.
10. **Reproduces the published baseline exactly** (9.62% EER), so our numbers are comparable with prior work.
11. **It uses many evidence types together.** Typing, automation, amount, device and payee each catch a different attack. A takeover with a new payee and a large amount is blocked, while a customer who only types differently today is only asked for a passkey.

17. Where this project is weak

1. **The deployed typing signal is weak.** With 9 summary features, the EER is 20.4%. One in five decisions on typing alone is wrong at the equal error point. This is the main reason typing can never block alone.
2. **The global threshold is loose.** Before any poisoning, 46% of an impostor's sessions pass the typing check without a step-up. This is the largest weakness in the identity channel.
3. **The poisoning defence beyond the passkey gate is small.** P3 is only about 8 points better than P2, and the intervals overlap. The budget does not limit an attacker who holds the passkey. We have not yet found a rule that fixes this.
4. **Learning itself loosens the profile.** Even with no attacker, an adapted profile accepts more impostors (60%) than a static profile (46%).
5. **Most attack results are synthetic (Tier 2).** The 100% detection rates come from our own simulator. Real attackers are more varied.
6. **The dataset is small and narrow.** 51 people, one 10-character password, 8 sessions, desktop keyboards only. "Drift" in CMU is mostly practice over a few days, not months of real change. The

phone and tablet profiles have no real-data test.

7. **The latency target is missed at p95.** 229 ms against a 200 ms target for the round trip. Cold starts are about 1.2 to 1.5 s.
8. **Siphoning detection is late.** Money moves before the flag (₹2,700 in the test). A siphoner whose typing is similar to the victim is not detected.
9. **Malware on the customer's own device, to a known payee, defeats every channel.** The device is known and the payee is known, so no channel sees anything unusual.
10. **Shared accounts.** Two genuine people on one account look like a takeover. The two-typist detector exists but is not in the live system.
11. **Scams are out of scope.** If a criminal tricks the real customer into a transfer, this system sees a genuine customer and allows it.
12. **Not every service is always free.** Step Functions, KMS and X-Ray charge for each use. The cost is small, but it is not zero.
13. **An attacker with the passkey has a lot of power.** The whole trust model rests on the passkey. If an attacker steals or controls the passkey device, they can pass step-ups and poison the profile.

18. Work for the future

- Find an anchor rule that limits an attacker who holds the passkey. Two ideas: move the anchor only on step-ups with a minimum time between them, or cap the total anchor movement in 30 days.
- Test on free-text typing datasets (for example Buffalo, Clarkson) and multi-device datasets (BB-MAS), to measure real drift and phone typing.
- Recalibrate the global behaviour threshold with real, not synthetic, sessions.
- Put the two-typist detector into the live system. This needs each session's features in the audit lake.
- Add graph features for mule networks (two-hop fan-in, group size).

19. How the code is arranged

```
src/fraudcore/      All decision logic. Pure Python. Shared by the cloud and the research.
  features.py       The 12 typing features and the other channel features
  scoring.py        Profiles, scaled Manhattan distance, the five channel scores
  fusion.py         Logistic combination, reasons, confidence
  policy.py         Thresholds, friction ceiling, corroboration, fail open
  adaptation.py     Trust, buffer, geometric median, displacement budget
  batch.py          Nightly calculations: aggregates, siphoning, two typists
  models.json       The fitted weights, thresholds and adaptation settings
src/lambdas/        Thin cloud functions. They read the request, call fraudcore, save the
result.
client/             The banking website, the capture script, the analyst console
research/           Offline experiments on CMU data (numpy is allowed only here)
simulator/          Attack traffic against the live system; latency and cost measurement
infra/              Terraform for the full AWS stack
tests/unit/         Fast tests, no AWS
tests/integration/  One test for each phase, against the deployed stack
```

The most important rule: **a cloud function contains no decision maths.** If it does, the research numbers no longer describe the deployed system.

Common commands:

Command	What it does
`make test`	Runs the unit tests.
`make test-integration`	Runs the integration tests against the deployed stack.
`make build`	Builds the Lambda packages.
`terraform plan` / `apply` in `infra/envs/dev`	Shows and makes changes to the AWS stack.

For a live demonstration, follow `docs/demo.md`.

20. Summary

This project checks bank transfers for fraud with five types of evidence: how the person types, whether the typing is a robot or a copy, whether the amount is unusual, whether the device or account details changed, and whether the payee is new or suspicious. It combines them with a small, exact formula into a score, a reason and an action. Fixed rules make sure that typing alone never blocks a customer and that a failure never blocks a customer. Money moves only when a workflow releases it.

The system runs serverless on AWS for 0.23 USD per thousand sessions, with no cost when idle and a typical response in about 145 ms.

The research question is how a typing profile can learn without an attacker poisoning it. On real typing data, the answer is: naive learning is easy to poison, a profile that never learns rejects too many genuine customers, and in the deployed system the passkey step-up gate is what really stops poisoning. The robust median and the movement budget add only a small extra defence. This is an honest result, and it shows clearly where the next work must go.